

Mediator

1. Introducción

2. Problema

3. Participantes

1. Mediator (`CitaMediator`)
2. Mediator Concreto (`CitaMediatorConcreto`)
3. Componentes Colegas

4. Diagrama UML

5. Código

- 5.1. Clase `CitaMediator` (Interfaz)
- 5.2. Clase `EquipoMedicoMediator` (Abstracción)
- 5.3. Variables en la clase Equipo Médico
- 5.4. Uso del código

6. Conclusión

1. Introducción

El **Patrón Mediator** es un patrón de comportamiento que permite reducir las dependencias entre objetos al hacer que se comuniquen a través de un objeto mediador central. De esta manera, se promueve un sistema más flexible, fácil de mantener y escalar.

En esta implementación, aplicamos el **Patrón Mediator** para **coordinar la gestión de citas médicas**, delegando la comunicación entre componentes como sucursales, médicos, fechas y horarios a un **Mediator Central**.

2. Problema

Inicialmente, los distintos componentes del sistema de citas (como módulos de médicos, sucursales y horarios) se comunicaban entre sí directamente, generando un alto **acoplamiento**. Esto hacía que:

- Cada cambio en un componente afectara a los demás.
- Se dificultara la reutilización y prueba de los módulos por separado.

- La lógica de interacción estuviera dispersa, complicando el mantenimiento.

Para solucionar esto, se introdujo el **Patrón Mediator**, centralizando la lógica de interacción en una única clase.

3. Participantes

1. Mediator (**CitaMediator**)

- Define la interfaz que utilizan los componentes para comunicarse entre sí.
- Gestiona la lógica de interacción sin que los componentes se conozcan directamente.

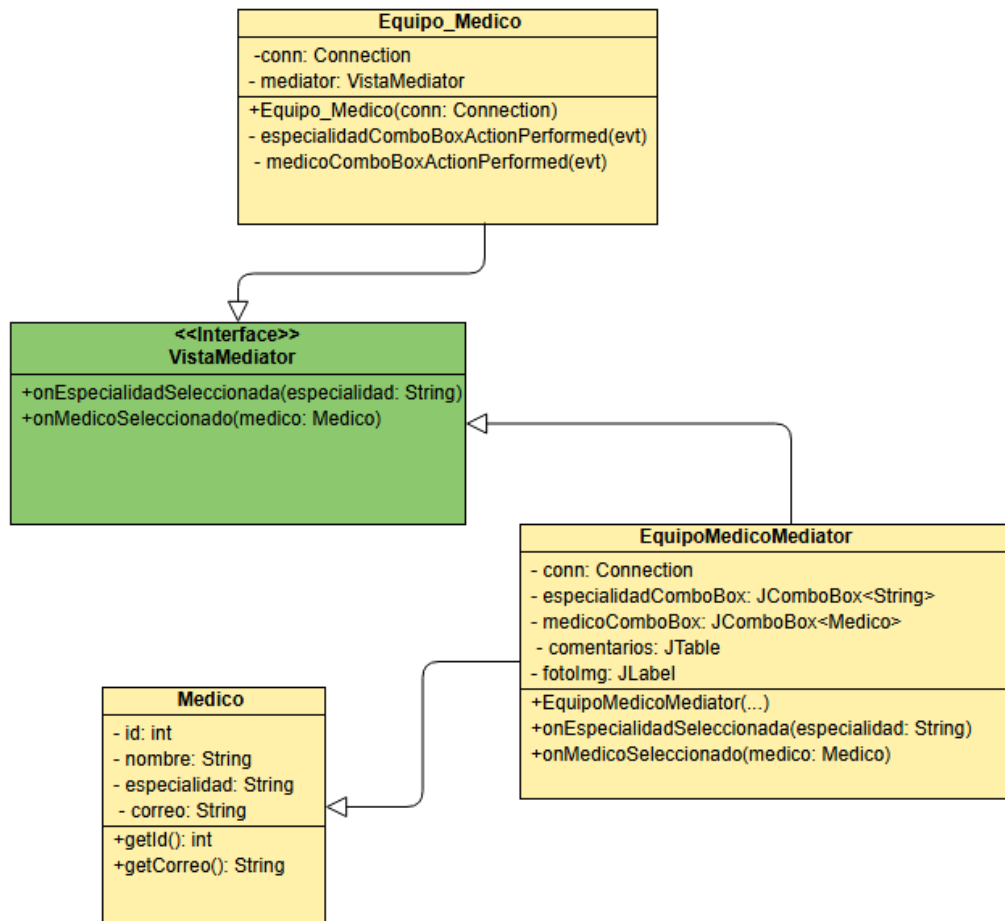
2. Mediator Concreto (**CitaMediatorConcreto**)

- Implementa la lógica de coordinación entre componentes como médicos, sucursales, fechas y horarios.
- Responde a las solicitudes de los módulos y canaliza la información.

3. Componentes Colegas

- **ModuloMedico** : Solicita fechas y horarios disponibles.
- **ModuloSucursal** : Informa sobre las sucursales disponibles.
- **ModuloCita** : Coordina el proceso completo de agendar una cita.
- Todos estos componentes se comunican únicamente a través del mediador.

4. Diagrama UML



5. Código

El código sigue la estructura del **Patrón Bridge**, donde `FormatoCita` es la abstracción y `FormatoCitaBD` es la implementación que creamos.

5.1. Clase `CitaMediator` (Interfaz)

```

public interface VistaMediator {
    void onEspecialidadSeleccionada(String especialidad);
    void onMedicoSeleccionado(Medico medico);
}

```

5.2. Clase **EquipoMedicoMediator** (Abstracción)

Esta clase recibirá los eventos de cambio y realizará las acciones necesarias.

```
public class EquipoMedicoMediator implements VistaMediator {
    private Connection conn;
    private javax.swing.JComboBox<String> especialidadComboBox;
    private javax.swing.JComboBox<Medico> medicoComboBox;
    private javax.swing.JTable comentarios;
    private javax.swing.JLabel fotoImg;

    public EquipoMedicoMediator(Connection conn,
                                javax.swing.JComboBox<String> especialidadComboBox,
                                javax.swing.JComboBox<Medico> medicoComboBox,
                                javax.swing.JTable comentarios,
                                javax.swing.JLabel fotoImg) {
        this.conn = conn;
        this.especialidadComboBox = especialidadComboBox;
        this.medicoComboBox = medicoComboBox;
        this.comentarios = comentarios;
        this.fotoImg = fotoImg;
    }

    @Override
    public void onEspecialidadSeleccionada(String especialidad) {
        try {
            medicoComboBox.removeAllItems();
            String query = "SELECT id, nombre, especialidad, correo FROM especi
alistas WHERE especialidad = ?";
            try (PreparedStatement stmt = conn.prepareStatement(query)) {
                stmt.setString(1, especialidad);
                ResultSet rs = stmt.executeQuery();
                while (rs.next()) {
                    Medico m = new Medico(rs.getInt("id"), rs.getString("nombre"), e
specialidad, rs.getString("correo"));
                    medicoComboBox.addItem(m);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}

@Override
public void onMedicoSeleccionado(Medico medico) {
    if (medico == null) return;

    DefaultTableModel model = new DefaultTableModel();
    model.addColumn("Comentarios");
    comentarios.setModel(model);

    String query = "SELECT opinion FROM opinionesequipomedico WHERE i
d_medico = ?";
    try (PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setInt(1, medico.getId());
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            model.addRow(new Object[]{rs.getString("opinion")});
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    try (Statement stmt = conn.createStatement()) {
        String imgQuery = "SELECT img FROM usuarios WHERE id = " + Regist
rar_Informe.getIdMedico2(conn, medico.getCorreo());
        ResultSet rs = stmt.executeQuery(imgQuery);
        if (rs.next()) {
            String rutaImg = rs.getString("img");
            if (rutaImg != null) {
                Image foto = java.awt.Toolkit.getDefaultToolkit().getImage(rutaIm
g);

```

```

        fotoImg.setICon(new ImageIcon(foto));
    } else {
        fotoImg.setICon(null);
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

5.3. Variables en la clase Equipo Médico

Proporciona la lógica concreta para acceder a los datos de la base de datos.

```

private VistaMediator mediator;

this.mediator = new EquipoMedicoMediator(conn, especialidadComboBox, m
edicoComboBox, comentarios, fotoImg);
cargarEspecialidades();

```

5.4. Uso del código

En los métodos

```

private void especialidadComboBoxActionPerformed(java.awt.event.ActionEv
ent evt) {
    mediator.onEspecialidadSeleccionada((String) especialidadComboBox.getS
electedItem());
}

private void medicoComboBoxActionPerformed(java.awt.event.ActionEvent e
vt) {
    mediator.onMedicoSeleccionado((Medico) medicoComboBox.getSelectedIt
em());
}

```

```
}
```

6. Conclusión

El uso del **Patrón Mediator** permitió centralizar la lógica de interacción entre los distintos módulos del sistema de citas médicas. Esto trajo beneficios claros:

- **Reducción del acoplamiento** entre clases, mejorando la mantenibilidad.
- **Flexibilidad para agregar o modificar módulos** sin afectar a los demás.
- **Mejor organización del flujo de comunicación** al delegar las responsabilidades de coordinación al mediador.

Este patrón resulta ideal para sistemas donde múltiples componentes deben colaborar entre sí sin depender directamente unos de otros.